

Research Agent

Abstract

LLM 기반 에이전트의 성능은 최근 빠르게 고도화되고 있으며, 특히 코딩과 수학처럼 정답 구조가 비교적 명확한 닫힌 환경에서는 이미 인간 전문가에 근접한 문제 해결 능력을 보이고 있다. 특히, GPT-5.4 계열 모델은 reasoning, coding, 장문맥 처리, 도구 사용, 그리고 다단계 agentic workflow 수행 능력에서 뚜렷한 개선을 보였으며, 이는 문제 파악, 해결 전략 수립, 실험 설계로 이어지는 연구 과정의 상당 부분을 모델 중심으로 조직할 수 있음을 시사한다. 예를 들어, Karpathy의 autoresearch는 코드 수정, 단기 학습, 검증, 채택/폐기를 반복하는 자율 실험 루프를 보여주었고, 수학 영역에서도 Erdős Problems 데이터베이스의 일부 open question에 대해 semi-autonomous 해결 사례가 보고되었다. 이러한 흐름을 바탕으로, 문제 파악과 해결 전략 제안은 강력한 reasoning 모델이 담당하고, 실제 실험 수행과 결과 검증은 시스템 에이전트(codex 등)가 담당하는 자율 연구 에이전트용 harness를 제안한다. 제안하는 harness는 문제 정의, 가설 생성, 실험 계획, 코드 수정, 실행, 평가, 결과 기록의 전 과정을 반복 가능한 루프로 구성하며, 강력하고 범용적인 reasoning 모델은 계획과 의사결정을 담당하고 시스템 에이전트는 실행 안정성, 재현성, 로그 추적, 실패 복구를 담당하도록 설계된다. 이를 통해 본 프레임워크는 상위 reasoning 모델의 추론 능력을 실제 실험 자동화와 연결하면서도, 연구 과정 전반의 신뢰성과 통제 가능성을 함께 확보하는 것을 목표로 한다.

1 Workflow

1.1 Overview

본 프레임워크는 토론(debate)과 실행(execution)을 번갈아 수행하는 순환형 2단계 제어 루프로 구성된다. 각 사이클 c 에서 시스템은 원래 문제 설명, 이전 사이클에서 누적된 요약, 그리고 최신 실험 결과를 바탕으로 사이클별 문제 문맥 P_c 를 구성한다. 최상위 제어기는 먼저 debate section을 호출하여 후보 연구 방향을 생성하고 검토한 뒤, experiment section을 호출하여 선택된 방향을 실행 가능한 코드, 단기 ablation, 장기 main run, 그리고 최종 요약으로 진행한다.

Algorithm 1는 사이클 수준의 전체 제어 절차를 보여준다. 각 사이클은 현재의 문제 문맥에서 시작하여 debate section과 experiment section을 순차적으로 수행한다. 실험 단계의 출력은 단순한 종료 보고서로 사용되지 않고, 다음 사이클의 초기 문맥을 구성하는 요약으로 압축된다. 이를 통해 프레임워크는 실패 사례, 부분적 성공, 구현 과정에서 축적된 지식을 다음 반복에 반영하는 제한된 형태의 iterative research를 수행할 수 있다.

Algorithm 1 Overall framework

Require: Bootstrap context P_c , auto-cycle count C , debate max rounds R , pre-main ablation rounds A , main-run recovery retries K

- 1: **for** $c = 1$ to C **do**
- 2: run `debate_section`
- 3: run `experiment_section`
- 4: **end for**
- 5: Transition to `WAITING_FOR_NEXT_PROBLEM`

1.2 Debate section

Debate section의 목적은 단순한 자유 형식의 토론을 수행하는 것이 아니라, 강력하고 보다 범용적인 모델(ChatGPT - pro extended thinking)을 주요 추론 도구로 사용하여 원시적인 문제 서술을 실제 실험으로 옮길 수 있는 계획으로 변환하는 데 있다. 이를 위해 프레임워크는 사이클별 debate log인 $\mathcal{D}_c$ 를 컨텍스트로 명시적으로 유지하며, 모든 제안, 비판, 수정, 의사결정 과정을 기록한다. 이 단계는 먼저 병렬 ideation으로 시작하여 초기 탐색의 다양성을 확보하고, 이후 narrowing 단계를 통해 여러 제안 중 실현 가능성이 높은 소수의 방향으로 압축한다. 그 다음 라운드에서는 criticism과 refinement를 반복하면서 실현 가능성, 식별 가능성, 계산 비용, 구현 범위, 평가 설계 측면의 약점을 본격적인 코드 수정 이전에 드러내도록 한다. Algorithm 2에서 전체 알고리즘을 확인할 수 있다.

여기서 `literature_surveyor` 누적된 debate trace를 바탕으로 현재 제안과 가장 밀접한 선행연구를 정리하여 debate log에 주입하는 역할을 한다. `critic` 모듈은 누적된 debate trace를 바탕으로 현재 제안의 결함, 누락된 가정, 취약한 연결고리를 지적하고, `refinement` 모듈은 그 비판에 대응하여 방향을 수정한다. 이후 final narrowing 단계는 갱신된 상태를 다시 압축하여 실행 가능한 수준의 실험 계획으로 정리한다. `action_decider`는 현재까지 축적된 근거가 실제 실험을 시작하기에 충분한지, 탐색 폭을 넓히기 위해 ideation을 다시 열어야 하는지, 혹은 추가 토론이 필요한지를 판단한다.

1.3 Experiment section

실행 단계에 진입하면 선택된 계획은 고수준 reasoning과 저수준 구현 사이를 연결하는 구조화된 중간 산출물로 변환된다. Experiment section은 pseudocode specification과 draft note를 먼저 작성하고, 이후 main experiment code를 이 중간 표현에 맞게 정렬한다. 이는 실험 절차의 의도를 점검 가능하게 만들고, 계획과 구현 사이의 드리프트를 줄이며, 시스템 에이전트가 이후 코드 수정에서 참조할 명시적인 목표를 제공한다. 즉, 느슨한 자연어 목표를 곧바로 실행 환경에 전달하는 대신, 고비용 실행 전에 실험 의도를 더 명확한 표현으로 고정하기 위함이다.

Algorithm 3은 이후의 staged execution 절차를 보여준다. 장시간 main run을 시작하기 전에 프레임워크는 A 회의 짧은 ablation을 수행하여 불안정한 하이퍼파라미터를 조기에 식별하고, 명백한 구현 오류를 발견하며, 기본 설정값에 대한 경험적 근거를 확보한다. 이 단계의 결과는 tuning artifact로 정리되며, 이는 최종 main run의 설정을 제약하는 역할을 한다. 여기서 pre-main 단계의 목적은 성능을 끝까지 최적화하는 것이 아니라, 피할 수 있는 실패 모드를 줄이고 이후의 고비용

실험이 보다 안정적으로 수행되도록 만드는 데 있다.

long main run은 가장 비용이 큰 단계로 간주되므로, 명시적인 recovery 메커니즘과 함께 운용된다. 실행이 실패했을 때 시스템은 단순히 처음부터 재시작하지 않고, 먼저 raw failure artifact를 분석하고 recovery note를 작성한 뒤, 이 실패가 재시도 가능한 유형인지와 재실행이 정당화되는지를 판단한다.

실험의 마지막 단계에서 프레임워크는 문제 문맥 P_c , debate trace인 $\mathcal{D}_c$, tuning artifact, main-run output, 그리고 recovery log 전체를 종합적으로 분석하고, 그 결과를 experiment summary로 압축한다. 이 summary는 사람이 읽을 수 있는 기록이자 다음 사이클의 초기 문맥으로 재사용되는 기억 단위로 기능한다. 따라서 시스템은 매번 전체 상호작용 이력을 그대로 재생하지 않고도, 핵심 상태를 축적하면서 반복적인 연구를 수행할 수 있다.

Algorithm 2 `debate_section`

Require: Bootstrap context P_c , auto-cycle count C , debate max rounds R ,
pre-main ablation rounds A , main-run recovery retries K

- 1: Resolve problem context P_c
- 2: Check external LLM readiness
- 3: Initialize debate log $\mathcal{D}_c$
- 4: Scribe initial problem context into $\mathcal{D}_c$
- 5: Run `idea_generator_1` and `idea_generator_2` in parallel
- 6: Scribe both idea outputs into $\mathcal{D}_c$
- 7: Run `literature_surveyor` on the accumulated debate log
- 8: Scribe literature survey output into $\mathcal{D}_c$
- 9: Run `narrowing` on the accumulated debate log
- 10: Scribe narrowing output into $\mathcal{D}_c$
- 11: **for** $r = 1$ to R **do**
- 12: Run `critic` on the accumulated debate log
- 13: Scribe critic output into $\mathcal{D}_c$
- 14: Run `refinement` on the accumulated debate log
- 15: Scribe refinement output into $\mathcal{D}_c$
- 16: Run `final_narrowing` on the accumulated debate log
- 17: Scribe final narrowing output into $\mathcal{D}_c$
- 18: Run `action_decider` on the accumulated debate log
- 19: **if** `action_decider` returns `START_EXPERIMENT` **then**
- 20: **break**
- 21: **else if** `action_decider` returns `REOPEN_IDEATION` **then**
- 22: Run `idea_generator_1` and `idea_generator_2` in parallel
- 23: Scribe both idea outputs into $\mathcal{D}_c$
- 24: Run `literature_surveyor` on the accumulated debate log
- 25: Scribe literature survey output into $\mathcal{D}_c$
- 26: Run `narrowing` on the accumulated debate log
- 27: Scribe narrowing output into $\mathcal{D}_c$
- 28: **else if** `action_decider` returns `CONTINUE_DEBATE` **then**
- 29: **pass**
- 30: **end if**
- 31: **if** $r = R$ and no `START_EXPERIMENT` **then**
- 32: Terminate
- 33: **end if**
- 34: **end for**

Algorithm 3 experiment_section

Require: Bootstrap context P_c , auto-cycle count C , debate max rounds R ,
pre-main ablation rounds A , main-run recovery retries K

```
1: Enter experiment workflow using  $P_c$  and  $\mathcal{D}_c$ 
2: Write pseudocode.tex, draft.tex from  $P_c$  and  $\mathcal{D}_c$ 
3: Modify main experiment code as pseudocode.tex
4: for  $a = 1$  to  $A$  do
5:   Run codex preparation / short ablation round  $a$ 
6:   Read all prior ablation logs from the same cycle
7:   if  $a = A$  then
8:     Write tuning.md
9:     Apply selected defaults directly in training code
10:  end if
11: end for
12: for  $t = 1$  to  $K + 1$  do
13:   Run the bare long main experiment
14:   if main run succeeds then
15:     break
16:   end if
17:   if  $t \leq K$  then
18:     Run codex recovery on the failed raw artifact
19:     Write recovery_##.md
20:     Read all prior recovery logs from the same cycle
21:     if recovery is not retryable or not relaunch-ready then
22:       break
23:     end if
24:   end if
25: end for
26: Run codex analysis on problem context, debate log, tuning log, main-run
   artifact, and recovery logs
27: Write final experiment summary experiment.md
28: if the cycle ends in failure then
29:   stop
30: end if
31: Save latest experiment summary as the next-cycle context seed
32: Transition to WAITING_FOR_NEXT_PROBLEM
```
